

INDUSTRIAL AUTOMATION SERIES

Wireshark

for SCADA Engineers

Protocol capture, display filters, and dissector analysis

Modbus, DNP3, and OPC UA on the wire

Security anomaly detection and field forensics

10 Chapters · Filter Reference · Field War Stories · tshark Cheatsheet

For SCADA and Industrial Automation Engineers

Contents

1	Introduction to Wireshark	1
1.1	What Wireshark Is	1
1.2	History	1
1.3	The Three-Pane Interface	2
1.4	Wireshark vs. tshark vs. dumpcap	3
1.5	Supported Protocols Relevant to SCADA	3
2	Capture Techniques & Interfaces	5
2.1	The Problem With Switched Networks	5
2.2	SPAN Port Configuration	6
2.3	Network Taps	6
2.4	Promiscuous Mode and Monitor Mode	6
2.5	Capture Interfaces	7
2.6	Capture File Management	7
2.7	pcap vs. pcapng Format	8
3	Display & Capture Filters	9
3.1	Capture Filters vs. Display Filters	9
3.2	Display Filter Syntax	9
3.3	Essential SCADA Display Filters	10
3.4	BPF Capture Filter Syntax	11
3.5	Following a TCP Stream	11
3.6	Colorization Rules	12
4	Protocol Dissectors & Decode As	13
4.1	What a Dissector Does	13
4.2	Automatic Protocol Detection	13
4.3	Decode As	13
4.4	Wireshark Profiles	14
4.5	Custom Lua Dissectors	14
4.6	Statistics Tools	15

5	Analyzing Modbus on the Wire	17
5.1	The Critical Security Reality	17
5.2	Modbus TCP Frame Structure	17
5.3	Reading a Modbus Capture in Wireshark	18
5.4	Identifying Modbus Exceptions	18
5.5	Spotting Unauthorized Write Commands	19
5.6	Using Statistics → I/O Graph for Modbus	20
6	Analyzing DNP3 Traffic	21
6.1	DNP3 on the Wire	21
6.2	DNP3 Frame Structure	21
6.3	Key DNP3 Analysis Scenarios	22
6.4	Secure Authentication v5 in Wireshark	23
6.5	DNP3 Capture Filters	24
7	Analyzing OPC UA Sessions	25
7.1	OPC UA Wire Transport	25
7.2	OPC UA Session Lifecycle	25
7.3	The Security Mode Problem	25
7.4	Subscription Traffic	26
7.5	OPC UA Statistics Workflow	26
8	Security Analysis & Anomaly Detection	29
8.1	The Baseline Principle	29
8.2	Anomaly Detection Workflow	29
8.3	Expert Information	30
8.4	Modbus Write Security Analysis	31
8.5	Detection Signatures for tshark	31
8.6	Incident Response: What to Capture	31
9	Advanced Wireshark Techniques	33
9.1	tshark: Command-Line Wireshark	33
9.2	Remote Capture	34
9.3	Packet Editing and Replay	34
9.4	Lua Scripts for Automation	35
9.5	Name Resolution and GeoIP	36
9.6	Wireshark Profiles for ICS Environments	36
10	Protocol Analysis Lab	37
10.1	Lab 1: Identify a Misconfigured Poll Rate	37
10.2	Lab 2: Trace an Exception Code 02 Storm	37
10.3	Lab 3: Verify OPC UA Security Mode	38
10.4	Lab 4: Detect a Rogue Modbus Master	38

10.5 Lab 5: Build a Monitoring Script	39
10.6 Filter Quick-Reference Card	39
10.7 tshark Quick-Reference Card	40
Recommended Reading	43
Certification Context	45

Chapter 1

Introduction to Wireshark

1.1 What Wireshark Is

Wireshark is a network protocol analyzer — a tool that captures raw frames off a network interface, decodes them according to protocol specifications, and presents the result in a form a human can actually read. It was written by Gerald Combs in 1998, originally under the name Ethereal, and renamed in 2006 when Combs left the company that held the trademark. The software is open source, cross-platform, and free. There is no commercial license required to run it on your industrial network.

The stethoscope analogy is exact: a doctor who guesses at a diagnosis without listening to the patient is a liability. An engineer who guesses at a protocol fault without looking at the wire is doing the same thing. Wireshark ends the guessing.

What Wireshark Actually Does

1. **Captures** raw Ethernet frames (or serial frames via adapter) from a network interface.
2. **Decodes** each frame using protocol dissectors — built-in or custom code that knows the byte layout of a given protocol.
3. **Displays** the result in three panes: packet list, packet detail, and raw hex dump.
4. **Filters** traffic using either capture filters (BPF syntax, applied before capture) or display filters (Wireshark syntax, applied post-capture).

1.2 History

Gerald Combs wrote Ethereal in 1998 as a University of Missouri project. The tool grew into the de facto standard for protocol analysis. In 2006, Combs left CACE Technologies (the corporate sponsor) and took the codebase with him. Unable to

take the “Ethereal” name — which was trademarked by CACE — he renamed it Wireshark. The rename happened over a single weekend. Most of the developer community followed him.

The pcap capture library (libpcap on Unix, WinPcap/Npcap on Windows) was developed independently and predates Wireshark. The pcap file format became the universal exchange format for packet captures — every major security tool reads it without conversion.

Field Note

The Ukraine power grid attack of December 2015 involved ICS protocols leaving forensic evidence in network captures. Investigators reconstructed the entire attack sequence from PCAP files. The attacker used standard protocols — Modbus, a remote access tool, and VPN traffic — all of which Wireshark dissects natively. The investigation that took weeks would have been impossible without packet capture infrastructure in place before the attack.

1.3 The Three-Pane Interface

Packet List Pane (Top)

One row per captured frame. Default columns: frame number, timestamp, source address, destination address, protocol, length, info summary. The Info column is generated by the dissector and often contains the most useful at-a-glance data: “Modbus/TCP, Function 3, Read Holding Registers, addr=100, count=10.”

Packet Detail Pane (Middle)

The selected frame expanded into a tree of decoded protocol layers. Each layer corresponds to one protocol in the encapsulation stack. A Modbus TCP frame shows: Frame (Ethernet layer) → Internet Protocol → Transmission Control Protocol → Modbus/TCP. Expanding any node reveals individual fields with their decoded values and byte positions.

Packet Bytes Pane (Bottom)

The raw hex dump of the captured frame, with the currently selected field in the detail pane highlighted. This pane is essential for verifying that the dissector is correct and for reading fields that the dissector does not decode.

Reading the Hex Dump

The bytes pane shows two columns: hexadecimal on the left, ASCII on the right. Non-printable bytes appear as dots. When you click a field in the packet detail pane, the corresponding bytes highlight in both columns. This is how you verify: “the dissector says Function Code = 3, and the raw byte at offset 7 is 0x03. Correct.”

1.4 Wireshark vs. tshark vs. dumpcap

Wireshark ships with several executables that are used in different contexts.

Tool	Purpose
wireshark	Full GUI. Interactive analysis.
tshark	Command-line version. Scriptable. Use for automated monitoring, alerts, and headless captures.
dumpcap	Capture-only. No analysis. Lower overhead. Used by wireshark/tshark as the underlying capture engine.
editcap	Edit or convert PCAP files: split by packet count, trim time ranges, convert formats.
mergcap	Merge multiple PCAP files into one.
capinfos	Print summary statistics about a PCAP file.

WARNING

Running Wireshark as root (or Administrator) to gain capture access is a security risk. The correct approach: on Linux, add your user to the wireshark group (`sudo usermod -aG wireshark $USER`) and restart the session. On Windows, Npcap installs a service that handles privileged capture. Wireshark itself should run as a normal user.

1.5 Supported Protocols Relevant to SCADA

Wireshark includes 3,000+ protocol dissectors. Of these, approximately 30 matter for industrial automation. The ones you will use most frequently:

Protocol	Filter Key	Notes
Modbus/TCP	<code>modbus</code>	Port 502. Full dissector since Wireshark 1.4.
DNP3	<code>dnp3</code>	Over TCP/UDP port 20000.
OPC UA Binary	<code>opcua</code>	Port 4840. Detects “OPC.TCP” magic bytes.
IEC 60870-5-104	<code>iec104</code>	Port 2404.
IEC 61850 MMS	<code>mms</code>	Port 102 (over COTP).
EtherNet/IP	<code>enip</code>	Port 44818 (TCP) / 2222 (UDP).
PROFINET	<code>pn_rt</code>	Ethernet-level, EtherType 0x8892.
BACnet/IP	<code>bacapp</code>	Port 47808 (UDP).
ICCP	<code>cotp</code>	Over ISO transport.
Secure Authentication v5	<code>dnp3</code>	Embedded in DNP3 frames.

Chapter 2

Capture Techniques & Interfaces

2.1 The Problem With Switched Networks

On a hub network, every device received every frame. You plugged in a laptop, set the NIC to promiscuous mode, and you saw everything. Modern networks use switches. A switch learns MAC addresses and forwards frames only to the port of the intended destination. Your laptop plugged into a switch port sees only frames addressed to its own MAC address, plus broadcasts and multicasts. You see almost nothing.

For Wireshark to capture traffic between two other devices on a switched network, you need one of three things: a SPAN port, a network tap, or ARP manipulation (not appropriate for production ICS networks).

Three Ways to Capture on a Switched Network

1. **SPAN port (Switch Port Analyzer):** Configure the switch to copy frames from one or more source ports to a dedicated mirror port. Connect your capture laptop to the mirror port. Works on managed switches. Every major vendor supports it under some name: SPAN (Cisco), Port Mirroring (HP/Aruba/Juniper).
2. **Network tap:** A physical device inserted in-line between two devices. Passes traffic normally, but splits the signal and outputs a copy to a capture port. Completely passive — the tap introduces no latency and cannot be disabled by software. The correct choice for long-term monitoring.
3. **ARP poisoning (man-in-the-middle):** Poisons the ARP cache of two devices to route traffic through your capture machine. **Do not do this on production ICS networks.** ARP poisoning can disrupt normal communications and cause control system faults.

2.2 SPAN Port Configuration

A SPAN session on a managed switch copies frames from source ports (or VLANs) to a destination port. Key constraints:

- The SPAN destination port receives copies of all frames from the source. It does not participate in normal switching while SPAN is active.
- SPAN has no impact on source port traffic if the switch has sufficient backplane bandwidth. On older or lower-end switches, heavy SPAN sessions can affect performance.
- Most switches support only one SPAN destination port.
- SPAN copies the frame as seen at the source port — including VLAN tags if configured for that.

Example: Cisco IOS SPAN Configuration

```
monitor session 1 source interface GigabitEthernet0/1
monitor session 1 destination interface GigabitEthernet0/24
```

This mirrors all traffic on Gi0/1 to Gi0/24. Connect Wireshark to Gi0/24.

2.3 Network Taps

A passive optical or copper tap breaks the physical signal path and outputs a copy to monitor ports. The original devices see no change in behavior. Power failure in the tap passes traffic through (failopen). Benefits over SPAN:

- **100% copy.** SPAN can drop packets under load. A passive tap does not.
- **Cannot be disabled remotely.** Physical device in the cable path.
- **No software configuration.** No risk of a misconfigured switch command affecting your capture.

Field note: A tap on the PLC-to-HMI link at a water treatment plant ran for 18 months before anyone looked at the data. The capture revealed that the HMI was polling 10× more frequently than configured — a client that had been misconfig-

2.4 Promiscuous Mode and Monitor Mode

Promiscuous mode tells the NIC to accept all frames, not just those addressed to its own MAC. In Wireshark, this is the default behavior and is what you want on a SPAN port or tap. Without promiscuous mode, you would only see frames sent to your own MAC address.

Monitor mode applies specifically to Wi-Fi adapters and allows capture of 802.11 management and control frames in addition to data frames. Not relevant for wired ICS captures but occasionally used for wireless SCADA links or WirelessHART.

2.5 Capture Interfaces

When Wireshark starts, it lists all available capture interfaces with a live throughput sparkline for each. Select the interface connected to your SPAN port or tap. On Windows, interfaces are named by GUID; on Linux, by standard names (eth0, ens33, etc.).

Quick Reference

- **Loopback capture:** Linux: lo. Windows: requires Npcap loopback adapter.
- **Any interface:** Linux supports capturing on “any” interface (all interfaces simultaneously). Windows does not.
- **Remote capture:** Capture on a remote machine and pipe to local Wireshark via rpcapd or SSH + dumpcap.
- **USB capture:** Requires USBPcap on Windows. Useful for USB-to-serial adapters running Modbus RTU.

2.6 Capture File Management

For long-duration captures, writing to a single file is impractical. Use a ring buffer: Wireshark writes to a fixed number of files, then overwrites the oldest.

Example: Ring Buffer Setup

In Wireshark: Capture → Options → Output tab.

Check “Automatically create a new file after” and set a size limit (e.g., 100 MB).

Check “Use a ring buffer with” and set file count (e.g., 10 files).

Result: maximum 1 GB of capture storage, always containing the most recent traffic.

Equivalent tshark command:

```
tshark -i eth0 -b filesize:102400 -b files:10 -w /captures/scada.pcap
```

Field Note

A \$15 Raspberry Pi Zero W running tshark in a ring buffer, placed on a tap at a critical PLC, gives you 24/7 capture capability with zero licensing cost. When an incident occurs, the PCAP is already there. Every OT environment with untapped critical links is running blind.

2.7 pcap vs. pcapng Format

Feature	pcap	pcapng
Multiple interfaces	No	Yes
Packet comments	No	Yes
Interface metadata	No	Yes
Name resolution	No	Yes (embedded)
Compatibility	Universal	Wireshark, Zeek, newer tools
File extension	.pcap	.pcapng

Use pcapng for new captures. Use pcap when sharing with tools that do not support pcapng. Wireshark reads both. editcap converts between formats.

Common Mistakes

- Capturing on the wrong interface — SPAN destination is Gi0/24 but you selected Gi0/1.
- Forgetting to enable promiscuous mode before capturing on a SPAN port.
- Using ARP poisoning on a live ICS network.
- Capturing to a small disk partition — ring buffer not configured — capture dies at disk full.
- Expecting a passive tap to show traffic in both directions on one port. Passive taps have separate TX and RX monitor ports; you need two capture interfaces or a merge.

Chapter 3

Display & Capture Filters

Wireshark has two entirely separate filter systems. They look similar but use different syntax and are applied at different stages.

3.1 Capture Filters vs. Display Filters

	Capture Filter	Display Filter
When applied	Before capture, in the kernel	After capture, in Wireshark
Syntax	BPF (Berkeley Packet Filter)	Wireshark filter language
Packets dropped	Yes — filtered packets are lost	No — all packets kept
Performance	Reduces capture file size	No effect on file size
Use case	Known traffic only, disk space limited	Interactive analysis

WARNING

Capture filters drop packets that do not match. If your capture filter is wrong, you lose the traffic you needed. On production ICS captures, capture with **no filter** (capture everything) and use display filters for analysis. Disk space is cheap; re-capturing an incident is impossible.

3.2 Display Filter Syntax

Display filters use a field-based language. The general form is:

`protocol.field operator value`

Comparison Operators

Symbol	Word	Example
<code>==</code>	<code>eq</code>	<code>ip.addr == 192.168.1.100</code>
<code>!=</code>	<code>ne</code>	<code>tcp.port != 80</code>
<code>></code>	<code>gt</code>	<code>frame.len > 1000</code>
<code><</code>	<code>lt</code>	<code>modbus.func_code < 10</code>
<code>>=</code>	<code>ge</code>	<code>tcp.port >= 502</code>
<code>contains</code>		<code>dns.qry.name contains "scada"</code>
<code>matches</code>		Regular expression match

Logical Operators

Symbol	Word	Example
<code>&&</code>	<code>and</code>	<code>ip.src == 10.0.0.1 && tcp.port == 502</code>
<code> </code>	<code>or</code>	<code>modbus dnp3</code>
<code>!</code>	<code>not</code>	<code>!arp</code>
<code>()</code>		<code>(modbus dnp3) && ip.src == 10.0.0.1</code>

3.3 Essential SCADA Display Filters

Filter Reference

Goal	Filter
All Modbus traffic	<code>modbus</code>
Modbus write operations	<code>modbus.func_code == 6 modbus.func_code == 15 modbus.func_code == 16</code>
Modbus exceptions	<code>modbus.exception_code > 0</code>
DNP3 traffic	<code>dnp3</code>
OPC UA traffic	<code>opcua</code>
Traffic to/from PLC	<code>ip.addr == 192.168.1.10</code>
Specific TCP conversation	<code>tcp.stream == 4</code>
Large frames (possible scan)	<code>frame.len > 1400</code>
ARP traffic	<code>arp</code>
Retransmissions	<code>tcp.analysis.retransmission</code>
Zero-window events	<code>tcp.analysis.zero_window</code>
Broadcast traffic	<code>eth.dst == ff:ff:ff:ff:ff:ff</code>

3.4 BPF Capture Filter Syntax

Berkeley Packet Filter syntax is used by `tcpdump`, `dumcap`, and most other capture tools. It is different from Wireshark display filter syntax.

Example: BPF Capture Filters

```
port 502           # Modbus TCP
port 20000         # DNP3
port 4840         # OPC UA
host 192.168.1.10  # Traffic to/from specific host
not port 22        # Exclude SSH management traffic
port 502 or port 20000 # Modbus or DNP3
dst host 192.168.1.1 # Only frames TO this host
net 10.0.0.0/8     # Entire subnet
```

3.5 Following a TCP Stream

Individual packets show you individual frames. For a complete conversation, right-click any packet in a TCP session and select “Follow → TCP Stream.” Wireshark reassembles and displays the full byte stream of the session, with bytes from each

direction color-coded. This is how you read an entire OPC UA session, a sequence of Modbus requests and responses, or a DNP3 data transfer from start to finish.

Follow TCP Stream vs. Individual Packets

The Follow TCP Stream view shows the reassembled application-layer data. It does not show TCP handshake, retransmissions, or TCP overhead. For protocol analysis, it is the right view. For TCP troubleshooting (looking at window sizes, retransmissions, and RTT), use the individual packet view with `tcp.*` filters.

3.6 Colorization Rules

Wireshark applies color to rows in the packet list based on configurable rules (View → Coloring Rules). Default rules highlight TCP errors in red, broadcasts in light blue, and so on. Add custom rules for your environment:

- Modbus write commands in yellow: filter `modbus.func_code >= 5 && modbus.func_code != 7`
- DNP3 unsolicited responses in orange: filter `dnp3.ctrl.uns == 1`
- OPC UA sessions with no security in red: `opcua.transport.type == 1` (identify “None” mode separately)

Common Mistakes

- Using BPF syntax in the display filter bar (turns red and is ignored).
- Using display filter syntax in the capture filter dialog (may appear to work but gives wrong results).
- Filtering too aggressively at capture time and discarding context frames needed for analysis.
- Confusing `ip.addr == x` (matches either source or destination) with `ip.src == x` (source only).

Chapter 4

Protocol Dissectors & Decode As

4.1 What a Dissector Does

A protocol dissector is code that knows the byte layout of a specific protocol and translates a raw byte sequence into named fields with decoded values. Without a dissector, Wireshark shows you hex. With a dissector, it shows you “Modbus/TCP, Transaction ID: 42, Unit ID: 1, Function: Read Holding Registers (3), Starting Address: 100, Quantity: 10.”

Wireshark selects dissectors using port numbers, EtherType values, and pattern matching. A frame arriving on TCP port 502 is handed to the Modbus/TCP dissector. A frame with EtherType 0x8892 is handed to the PROFINET dissector.

4.2 Automatic Protocol Detection

Wireshark’s protocol detection is heuristic. For well-known ports, detection is deterministic: port 502 = Modbus/TCP. For protocols that run on non-standard ports, Wireshark may not identify them correctly.

Common ICS scenarios where automatic detection fails:

- Modbus on a non-standard port (vendor custom implementation using port 1502 or 5020).
- DNP3 on a port other than 20000 (some vendors use 20001 or custom ports).
- OPC UA Binary on a port other than 4840 (common in vendor-specific configurations).

4.3 Decode As

When automatic detection fails, use “Decode As” (Analyze → Decode As, or right-click a packet). This forces Wireshark to apply a specific dissector to packets on a

given port or in a given TCP stream.

Example: Force Modbus Dissector on Port 1502

1. Right-click a frame that should be Modbus but shows as TCP data.
2. Select “Decode As.”
3. In the Current column, select “Modbus/TCP.”
4. Click OK. All frames on that port are now decoded as Modbus.

The Decode As setting persists for the session and can be saved to a profile.

4.4 Wireshark Profiles

A Wireshark profile stores your custom settings: coloring rules, column configuration, display filter toolbar shortcuts, Decode As rules, and preferences. Create a profile for each environment type (Modbus plant, DNP3 substation, OPC UA integration) and switch profiles when connecting to different capture sources.

Profiles are stored in your Wireshark personal configuration directory:

- Linux: ~/.config/wireshark/profiles/
- Windows: %APPDATA%\Wireshark\profiles\

4.5 Custom Lua Dissectors

Wireshark supports plugins written in Lua. A Lua dissector can decode a proprietary vendor protocol that has no built-in dissector. The Lua API provides access to the same field registration and tree-building infrastructure used by built-in C dissectors.

Example: Minimal Lua Dissector Skeleton

```
local proto_vendor = Proto("vendorproto", "Vendor Protocol")
local f_cmd = ProtoField.uint8("vendorproto.cmd", "Command", base.
    HEX)
proto_vendor.fields = {f_cmd}

function proto_vendor.dissector(buffer, pinfo, tree)
    pinfo.cols.protocol = "VENDOR"
    local subtree = tree:add(proto_vendor, buffer())
    subtree:add(f_cmd, buffer(0, 1))
end

local tcp_table = DissectorTable.get("tcp.port")
tcp_table:add(9999, proto_vendor)
```

Place this file in the Wireshark plugins directory and restart.

4.6 Statistics Tools

Wireshark's Statistics menu provides analysis tools beyond the packet list:

Tool	Use
Protocol Hierarchy	Shows percentage of total traffic by protocol. Identify unexpected protocols instantly.
Conversations	Lists all TCP/UDP/IP conversations with packet counts and byte totals.
Endpoints	Lists all network endpoints that appear in the capture.
I/O Graph	Plots packet rate or byte rate over time. Visualize polling intervals and traffic bursts.
TCP Stream Graph	RTT, window scaling, throughput graphs for a selected TCP connection.
Expert Information	Aggregates Wireshark's automatic anomaly detection notes.
Flow Graph	Ladder diagram of a conversation's packet exchange.

Field Note

The Protocol Hierarchy (Statistics → Protocol Hierarchy) is the first thing to run on any new capture. It tells you what is actually on the network in two seconds. On one site visit, this revealed that 34% of traffic was NetBIOS Name Service — a Windows file-sharing protocol that had no business touching the control network. The PLC vendor's Windows-based HMI was doing automatic network browsing every 60 seconds.

Chapter 5

Analyzing Modbus on the Wire

5.1 The Critical Security Reality

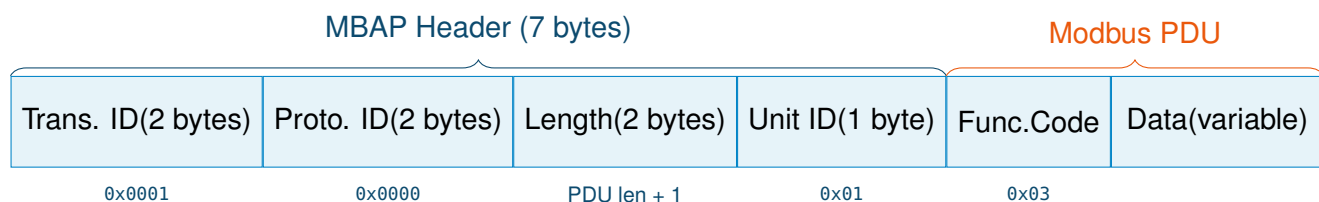
Modbus has no authentication. No encryption. No sequence numbers. No session concept. Any device with IP connectivity to a Modbus slave can send any function code to any register address, and the slave will execute it. Wireshark makes this viscerally real: every Modbus frame in your capture is readable by anyone who can capture the traffic, and any of those frames could have been sent by a malicious device rather than the legitimate master.

WARNING

Modbus TCP operates on port 502. The default configuration of most PLCs and field devices accepts connections from any IP address on port 502. There is no username. There is no password. “Secure” Modbus requires network segmentation, firewall rules, and protocol-aware inspection — none of which are in the Modbus specification itself.

5.2 Modbus TCP Frame Structure

Modbus TCP wraps the core Modbus PDU in a 7-byte MBAP header. The full frame:



MBAP Header Fields

- **Transaction ID:** Echoed in the response. Allows matching requests to responses in concurrent sessions.
- **Protocol ID:** Always 0x0000 for standard Modbus. Non-zero values indicate extensions or other protocols.
- **Length:** Number of remaining bytes, including Unit ID. Equal to (PDU length + 1).
- **Unit ID:** The slave device number. For TCP-only devices, this is typically 1 or 255. Used by Modbus gateways to route to the correct serial slave.

5.3 Reading a Modbus Capture in Wireshark

Apply display filter `modbus` to isolate Modbus traffic. The Info column shows the dissector summary. A normal polling sequence looks like:

No.	Source	Dest	Info
1	192.168.1.100	192.168.1.10	Modbus/TCP Query, Func: 3, Ref: 100, Cnt: 10
2	192.168.1.10	192.168.1.100	Modbus/TCP Response, Func: 3, BC: 20
3	192.168.1.100	192.168.1.10	Modbus/TCP Query, Func: 3, Ref: 110, Cnt: 10
4	192.168.1.10	192.168.1.100	Modbus/TCP Response, Func: 3, BC: 20

5.4 Identifying Modbus Exceptions

A Modbus exception response has Function Code = requested FC + 0x80, plus a one-byte exception code. In Wireshark, exception responses are flagged in the Info column. Filter: `modbus.exception_code > 0`.

Code	Name	Likely Cause
01	Illegal Function	Function code not supported by this slave
02	Illegal Data Address	Register address out of range; off-by-one addressing error
03	Illegal Data Value	Value out of range (e.g., quantity of 0)
04	Server Device Failure	Internal slave error
06	Server Device Busy	Slave processing long-running command

5.5 Spotting Unauthorized Write Commands

Function codes 5, 6, 15, and 16 are write operations. In a well-configured SCADA system, write commands come exclusively from authorized master devices to known register addresses during defined time windows. In Wireshark:

Filter Reference

```
modbus.func_code == 5    # Write Single Coil
modbus.func_code == 6    # Write Single Register
modbus.func_code == 15   # Write Multiple Coils
modbus.func_code == 16   # Write Multiple Registers
```

Or combined: `modbus.func_code == 5 || modbus.func_code == 6 || modbus.func_code == 15 || modbus.func_code == 16`

Check the source IP address of every write command. Any write from an IP that is not the authorized SCADA master is an anomaly requiring immediate investigation.

Field Note

During a security assessment of a water treatment plant, Wireshark captured Function Code 16 (Write Multiple Registers) writes to the dosing pump set-point register from an IP address that belonged to an engineering laptop. The laptop was connected to the process network “just for a quick config change” two weeks earlier and never disconnected. The laptop was compromised. The writes were being made by malware, not a human. The capture found it in 10 minutes.

5.6 Using Statistics → I/O Graph for Modbus

Open Statistics → I/O Graph. Add a second graph with filter `modbus.func_code >= 5` to plot write commands over time. A healthy system shows writes only during operator actions. Writes at 2 AM, writes in patterns inconsistent with human operation, or a sudden increase in write frequency are red flags that deserve investigation.

Chapter

Analyzing DNP3 Traffic

6.1 DNP3 on the Wire

DNP3 was designed for serial communications between substations and control centers in an era when eavesdropping required physical access to a copper cable. When DNP3 moved to Ethernet (DNP3 over TCP, RFC 4030-era practice), that physical security disappeared. Wireshark can capture a week of substation traffic from any compromised host on the same LAN segment. The protocol provides zero confidentiality or authentication in its base form.

DNP3 over TCP uses port 20000. Apply display filter `dnp3` to isolate it.

6.2 DNP3 Frame Structure

DNP3 uses a three-layer model:

1. **Data Link Layer:** Fixed header (10 bytes) with start bytes 0x0564, length, control byte, destination and source addresses, and CRC. Maximum Data Link frame: 292 bytes.
2. **Transport Layer:** Single byte prepended to each Data Link segment. Bit 7 (FIR) marks the first segment; Bit 6 (FIN) marks the last.
3. **Application Layer:** Contains the actual DNP3 function code, IIN bits, and object data.

Field	Value	Notes
Start bytes	0x0564	All DNP3 frames begin here
Length	1 byte	Bytes from Control to end of DL frame
Control	1 byte	Direction, master/slave, function
Destination	2 bytes	Little-endian address
Source	2 bytes	Little-endian address
CRC	2 bytes	Per 16-byte data block

6.3 Key DNP3 Analysis Scenarios

Identifying Unsolicited Responses

DNP3 outstation can be configured to send data without being polled (unsolicited reporting). In Wireshark, unsolicited responses have the Unsolicited bit set in the Application Control byte.

Filter Reference

```
dnp3.ctl.uns == 1
```

An outstation that has never been configured for unsolicited reporting but is sending unsolicited frames is behaving unexpectedly. This could indicate misconfiguration, a configuration change you were not told about, or a device sending spurious data.

Application Layer Function Codes

Code	Name	Direction
0x01	Read	Master → Outstation
0x02	Write	Master → Outstation
0x03	Select	Master → Outstation (SBO part 1)
0x04	Operate	Master → Outstation (SBO part 2)
0x81	Response	Outstation → Master
0x82	Unsolicited Response	Outstation → Master (unrequested)
0x83	Authenticate Request	SAv5 authentication

IIN Bits — Internal Indication Words

Every DNP3 response includes two IIN bytes that report the outstation's status. These are the first two bytes of the Application Layer in responses. Wireshark decodes them in the packet detail pane under "Application Layer" → "IIN".

Critical IIN Bits to Monitor

- **IIN1.6 (Device Trouble):** Internal hardware or firmware problem.
- **IIN1.5 (Config Corrupt):** Configuration data integrity failure.
- **IIN1.4 (Already Executing):** Operate received but prior Select not executed.
- **IIN2.5 (Config Changed):** Configuration was written since last Read.
- **IIN2.3 (Event Buffer Overflow):** Events lost because buffer filled before master polled.

Any of these in a Wireshark capture represents a condition that requires operator attention.

6.4 Secure Authentication v5 in Wireshark

DNP3 Secure Authentication version 5 (SAv5) adds challenge-response authentication between master and outstation. SAv5 frames appear in the DNP3 stream as Application Layer function codes 0x83 (Authentication Request) and 0x84 (Authentication Response).

Wireshark decodes SAv5 frames correctly. Look for the Authentication Challenge and Authentication Reply objects. If you see DNP3 traffic where critical control functions (Operate, Write) are being sent *without* preceding SAv5 challenge frames, the device is not using authentication for those commands.

Field Note

SAv5 was added to the DNP3 specification specifically because researchers demonstrated a man-in-the-middle attack using a laptop and Wireshark. The attack required no special hardware — capture a session, replay a valid Operate command from a different source. The subsequent SAv5 specification required cryptographic proof of identity for each critical operation. Wireshark remains the right tool for verifying that SAv5 is actually in use, not just configured.

6.5 DNP3 Capture Filters

Filter Reference

```
port 20000          # DNP3 TCP
dnp3                # All DNP3 (display filter)
dnp3ctl.uns == 1    # Unsolicited responses only
dnp3al.func == 4    # Operate commands only (SB0)
dnp3al.func == 2    # Write commands
```

Chapter 7

Analyzing OPC UA Sessions

7.1 OPC UA Wire Transport

OPC UA supports three transport mappings: OPC UA Binary (most common in production), OPC UA XML/SOAP (legacy), and OPC UA over WebSockets. Wireshark's OPC UA dissector handles binary transport, which operates on TCP port 4840. The dissector identifies OPC UA binary frames by the magic bytes 0x4F50432E544350 (OPC.TCP) at the start of the stream.

Apply display filter `opcua` to isolate OPC UA traffic.

7.2 OPC UA Session Lifecycle

An OPC UA session between client and server follows a defined sequence. Understanding this sequence lets you verify correct behavior in a Wireshark capture.

1. **Hello / Acknowledge:** Transport-level handshake. Client sends Hello message with receive buffer size. Server sends Acknowledge with its buffer parameters.
2. **OpenSecureChannel:** Establishes a secure channel. Security mode negotiated here: None, Sign, or SignAndEncrypt.
3. **CreateSession:** Application-level session created. Authentication credentials exchanged (username/password, certificate, or anonymous).
4. **ActivateSession:** Session activated. Client and server confirm identity.
5. **Normal operation:** Read, Write, CreateSubscription, PublishRequest/PublishResponse.
6. **CloseSession / CloseSecureChannel:** Clean teardown.

7.3 The Security Mode Problem

OPC UA defines three security modes:

- **None:** No signing, no encryption. All data transmitted in plaintext. The spec permits it. It should not be used in production.
- **Sign:** Messages are signed (integrity protected) but not encrypted. Content is readable.
- **SignAndEncrypt:** Messages are signed and encrypted. Content is not readable in the capture.

WARNING

Most OPC UA deployments in production use Security Mode None. The implementation guide said “sign and encrypt” but the commissioning engineer selected None for easier troubleshooting and the site never changed it back. Wireshark will show you exactly which mode is in use: look for the SecurityMode field in the OpenSecureChannel request frame.

Filter Reference

Filter for OpenSecureChannel requests to check security mode:

```
opcua
```

Then look in the packet detail for: OpcUa → Message → OpenSecureChannelRequest → RequestedLifetime / SecurityMode.

Security mode values: 0 = Invalid, 1 = None, 2 = Sign, 3 = SignAndEncrypt.

7.4 Subscription Traffic

In OPC UA, clients create Subscriptions and MonitoredItems. The server then sends PublishResponse messages when monitored item values change. A healthy subscription pattern:

- Client sends CreateSubscription (defines the publishing interval).
- Client sends CreateMonitoredItems (specifies which Node IDs to monitor).
- Server sends PublishResponse at the publishing interval, with changed values.
- Client sends Publish request to keep the subscription alive.

If you see PublishResponse messages arriving far more frequently than the configured publishing interval, the server is reporting changed values too rapidly — possible data flooding, misconfigured sampling interval, or a device reporting values with artificial jitter.

7.5 OPC UA Statistics Workflow

1. Apply `opcua` display filter.

2. Run Statistics → Protocol Hierarchy to confirm what fraction of traffic is OPC UA.
3. Run Statistics → Conversations to see all OPC UA TCP connections (source:port → destination:port).
4. For each connection, right-click and “Follow TCP Stream” to see the full session.
5. In packet detail, expand the OPC UA tree for each OpenSecureChannel to note security mode.

8

Chapter

Security Analysis & Anomaly Detection

8.1 The Baseline Principle

ICS network traffic is highly predictable. A Modbus master that has polled the same 10 registers every 5 seconds for three years does not suddenly need to poll 500 registers. A PLC that has never initiated a TCP connection outbound does not spontaneously connect to an external host. Wireshark-based security analysis works by establishing what normal looks like and flagging deviations.

What Makes ICS Traffic Predictable

- Fixed polling cycles: SCADA masters poll at configured intervals.
- Defined point lists: each master polls specific registers from specific devices.
- No browsing: legitimate ICS devices do not discover or probe new hosts.
- Known conversational pairs: the set of (source, destination) IP pairs in a healthy ICS network is small and stable.
- Time-bounded operations: maintenance windows, shift changes, batch starts.

Any traffic that violates these characteristics warrants investigation.

8.2 Anomaly Detection Workflow

Step 1: Establish Conversational Pairs

Statistics → Conversations → TCP tab. Export the list. Compare against your expected topology. Any (source, destination) pair not in the expected list is anomalous.

Step 2: Inventory Protocols

Statistics → Protocol Hierarchy. Unexpected protocols (NetBIOS, LLMNR, WSD, SMB on a control network) indicate devices behaving outside their defined role.

Step 3: Count Write Commands

Filter `modbus.func_code >= 5 && modbus.func_code <= 16` and check the frame count. Compare against historical baselines. A sudden increase in write commands without a corresponding operator action is a red flag.

Step 4: Check for Scanning

A host scanning a subnet makes sequential TCP connection attempts. Look for:

- One source IP connecting to many destination IPs (horizontal scan).
- One source IP connecting to many ports on one destination (vertical scan).
- Large numbers of TCP SYN with no corresponding SYN-ACK (RST storm from closed ports).

Filter: `tcp.flags.syn == 1 && tcp.flags.ack == 0`

Step 5: Check for Data Exfiltration

Look for large outbound data transfers from historian or HMI hosts to external destinations. Statistics → Conversations → sort by bytes descending. Any conversation with anomalous data volume from a control network host deserves inspection.

8.3 Expert Information

Wireshark automatically flags protocol-level anomalies in the Expert Information panel (Analyze → Expert Information). Severity levels:

- **Error (red):** Protocol violations — malformed frames, invalid field values.
- **Warning (yellow):** TCP retransmissions, out-of-order packets, window problems.
- **Note (blue):** Connection resets, keepalives, other protocol events.
- **Chat (gray):** Normal session events (connections established, terminated).

A capture with numerous TCP retransmissions on a control network link is a network reliability problem. Retransmissions introduce latency and can cause poll timeouts, which manifest as PLC alarms and control interruptions.

8.4 Modbus Write Security Analysis

Field Note

During a penetration test of a water treatment facility, the tester sent Modbus Function Code 16 (Write Multiple Registers) to the chlorine dosing pump setpoint. The test was authorized. Wireshark captured the entire sequence: connection, write command, the pump's register change reflected in the next poll response. The test took 45 seconds from laptop to changed setpoint. No authentication. No alarm. No operator notification. The only record was in the network capture — which had not been configured before the test.

Wireshark is not a firewall. It does not block writes. It is the evidence trail — the audit log that proves what commands were sent, from where, and when. For this reason, continuous packet capture with a ring buffer on critical ICS links is a security control, not just a troubleshooting tool.

8.5 Detection Signatures for tshark

tshark can run as a monitoring daemon and print alerts when specific conditions are met:

Example: Alert on Modbus Writes from Unexpected Source

```
tshark -i eth0 -Y "modbus.func_code >= 5 and ip.src !=
192.168.1.100" \
-T fields -e frame.time -e ip.src -e ip.dst -e modbus.func_code \
2>/dev/null | while read line; do
    echo "ALERT: Unauthorized Modbus write: $line" | mail -s "ICS
ALERT" ops@example.com
done
```

This runs indefinitely, printing (and emailing) any Modbus write command that does not come from the authorized master at 192.168.1.100.

8.6 Incident Response: What to Capture

When an ICS incident is suspected, capture immediately. Do not wait for approval workflows. The first thing to preserve:

1. Start a capture on the PLC's switch port (SPAN or tap).
2. Start a capture on the SCADA master's network interface.
3. Save existing ring buffer files before they rotate.
4. Note the exact timestamp (synchronized to GPS/NTP if available).
5. Do not reboot any device — volatile memory (ARP cache, routing tables, running processes) is evidence.

Chapter

Advanced Wireshark Techniques

9.1 tshark: Command-Line Wireshark

tshark provides all of Wireshark's analysis capability in a scriptable command-line interface. It is the correct tool for:

- Automated continuous monitoring.
- Capturing on headless servers and embedded systems.
- Post-processing large capture files with shell pipelines.
- Scheduled captures (via cron).

Quick Reference

Command	Purpose
<code>tshark -D</code>	List interfaces
<code>tshark -i eth0 -w out.pcap</code>	Capture to file
<code>tshark -r in.pcap</code>	Read PCAP file
<code>tshark -r in.pcap -Y "modbus"</code>	Display filter while reading
<code>tshark -r in.pcap -T fields -e ip.src -e modbus.func_code</code>	Extract specific fields
<code>tshark -r in.pcap -q -z io,phs</code>	Protocol hierarchy stats
<code>tshark -r in.pcap -q -z conv,tcp</code>	TCP conversation stats
<code>tshark -i eth0 -b filesize:51200 -b files:20 -w ring.pcap</code>	Ring buffer, 1 GB total

9.2 Remote Capture

Capturing on a remote machine and analyzing locally avoids the need to physically connect to a network tap.

SSH + tshark Pipe

Example: Remote Capture via SSH

```
ssh user@remote-host "tshark -i eth0 -w - -f 'port 502'" | wireshark -k -i -
```

This runs tshark on the remote host, writes raw pcap to stdout (the `-w -` flag), pipes it through SSH to the local machine, and feeds it directly into Wireshark (the `-k -i -` flags tell Wireshark to capture from stdin immediately). Live packet analysis from a remote network segment, no files on disk.

9.3 Packet Editing and Replay

`editcap` and `tcpreplay` enable PCAP manipulation:

- **Split a large capture:** `editcap -c 10000 big.pcap split.pcap` (10,000 packets per file).
- **Trim by time:** `editcap -A "2026-05-15 08:00:00" -B "2026-05-15 09:00:00" full.pcap trimmed.pcap`
- **Convert format:** `editcap -F pcapng input.pcap output.pcapng`
- **Replay traffic:** `tcpreplay -i eth0 capture.pcap` (replays at original rate; use `-multiplier` to speed up/slow down).

Replay is useful for testing IDS signatures against known-good captures and for verifying that a Wireshark alert would fire on real attack traffic.

9.4 Lua Scripts for Automation

Beyond custom dissectors, Lua scripts can automate analysis tasks. A Lua post-dissector runs after all built-in dissectors and can add new fields, modify the display, or write to external logs.

Example: Lua Script: Log All Modbus Writes to File

```
local logfile = io.open("/tmp/modbus_writes.log", "a")

local modbus_fc = Field.new("modbus.func_code")
local ip_src = Field.new("ip.src")
local frame_time = Field.new("frame.time")

local tap = Listener.new("modbus")

function tap.packet(pinfo, tvb)
    local fc = modbus_fc()
    if fc and (fc.value == 5 or fc.value == 6 or
              fc.value == 15 or fc.value == 16) then
        local src = ip_src()
        local t = frame_time()
        logfile:write(tostring(t) .. " " ..
                     tostring(src) .. " FC=" ..
                     fc.value .. "\n")
    end
end
```

Run with: `tshark -r capture.pcap -X lua_script:log_writes.lua -q`

9.5 Name Resolution and GeoIP

Wireshark resolves IP addresses to hostnames (DNS), MAC addresses to vendor names (OUI database), and can resolve geographic location via MaxMind GeoIP databases. In ICS environments:

- **Disable DNS resolution** for captures on isolated networks — DNS queries will fail and slow Wireshark.
- **Use custom host files:** Edit → Preferences → Name Resolution → “hosts” file path. Add your PLC and SCADA host names for readable packet lists.
- **GeoIP** is useful for identifying unexpected external connections from historian or HMI hosts.

9.6 Wireshark Profiles for ICS Environments

Create separate Wireshark profiles for each network type you work on. A “Modbus Plant” profile includes:

- Column: Modbus Function Code (`modbus.func_code`)
- Column: Unit ID (`modbus.unit_id`)
- Coloring rule: write commands (yellow)
- Coloring rule: exception responses (red)
- Custom host file with PLC and SCADA names
- Default display filter: `modbus`

Switch profiles at Wireshark startup or via Edit → Configuration Profiles.

Common Mistakes

- Using `-w` - without a consuming process (tshark writes to stdout, nothing reads it, capture fails).
- Forgetting `-q` with statistics commands — tshark prints every packet before the stats without it.
- Replaying traffic on a live production network (tcpreplay sends real frames; Modbus replays can trigger actual control actions).
- Not handling Lua errors — an unhandled Lua error silently stops the tap without warning.

Chapter 10

Protocol Analysis Lab

This chapter presents practical analysis scenarios. Each exercise uses a described capture scenario. Work through the analysis steps before reading the solution.

10.1 Lab 1: Identify a Misconfigured Poll Rate

Scenario: A SCADA engineer reports that a remote pump station PLC is generating excessive network traffic. The network team says traffic from the PLC's IP has doubled compared to last month. You have a 10-minute PCAP from the link.

Analysis steps:

1. Apply display filter `ip.src == [PLC IP]`.
2. Open Statistics → I/O Graph. Set Y-axis to "Packets/s." Note the rate.
3. Filter to `modbus && ip.src == [PLC IP]`. Count response frames.
4. Compare response rate to the expected poll cycle. If expected poll is 5 seconds (12/min) but actual is 0.5 seconds (120/min), poll rate was changed.
5. Check whether the client (SCADA master) or the PLC is initiating more frequently.

Root cause pattern: A software update to the SCADA client reset the poll interval from 5000 ms to 500 ms for this point set. The PLC was responding correctly to every request.

10.2 Lab 2: Trace an Exception Code 02 Storm

Scenario: A PLC's alarm log shows intermittent Modbus timeouts. The on-call engineer suspects cable faults. You start a capture.

Analysis steps:

1. Apply filter `modbus.exception_code == 2`.

2. For each exception frame, expand the request that preceded it (use the transaction ID to match request and response).
3. Note the register address in the failing request.
4. Compare the failing address against the PLC's register map documentation.

Root cause pattern: Exception Code 02 (Illegal Data Address) appears on requests for register 40501. The PLC documentation shows the valid range ends at 40500. The SCADA configuration was recently updated with an off-by-one error. No cable fault — the network is healthy. The cable call was cancelled, saving four hours of field work.

10.3 Lab 3: Verify OPC UA Security Mode

Scenario: A new OPC UA integration was deployed between a historian and a DCS. The integration specification requires SignAndEncrypt. You need to verify it is actually in use.

Analysis steps:

1. Apply display filter `opcua`.
2. Look for frames with message type "OpenSecureChannelRequest" in the Info column.
3. Click the frame. In the packet detail pane, expand: OpcUa → Message → OpenSecureChannelRequest → SecurityMode.
4. Record the SecurityMode value. 1 = None, 2 = Sign, 3 = SignAndEncrypt.
5. If mode = 1 or 2, the integration is not meeting the specification.

What correct traffic looks like: SecurityMode = 3, and subsequent application-layer messages are encrypted (Wireshark shows encrypted content, cannot decode application data). If you can read tag names and values in the OPC UA frames, the connection is not encrypted.

10.4 Lab 4: Detect a Rogue Modbus Master

Scenario: Operators report unexpected valve movements during a night shift. No operator was logged into the SCADA workstation at the time. You pull the capture from the ring buffer.

Analysis steps:

1. Filter: `modbus.func_code == 5 || modbus.func_code == 6 || modbus.func_code == 15 || modbus.func_code == 16`
2. Check the source IP of every write command during the time window.
3. Compare against the authorized master IP address.

4. If writes originate from an unexpected IP, run Statistics → Conversations and find all sessions involving that IP.
5. Trace the conversation timeline: did this IP connect to other devices? Did it scan?

Root cause pattern: Write commands at 02:14 AM originate from 192.168.1.55 — an engineering laptop that had been connected to the control network two weeks prior. The laptop was running automation software configured to write to a test PLC; the IP had been reused and now pointed at the production PLC. The writes were not malicious — they were accidental. The ring buffer capture resolved a four-hour investigation in 20 minutes.

10.5 Lab 5: Build a Monitoring Script

Objective: Write a tshark command that monitors a live interface for DNP3 Operate commands and logs them with timestamp, source, and destination.

Solution:

```
tshark -i eth0 \  
-Y "dnp3.al.func == 4" \  
-T fields \  
-e frame.time \  
-e ip.src \  
-e ip.dst \  
-e dnp3.dst \  
-E separator="," \  
>> /var/log/dnp3_operates.csv
```

Each line in the output CSV contains: timestamp, source IP, destination IP, DNP3 destination address. Run this in a screen session or as a systemd service. The file is your audit trail for every control-level command sent on the DNP3 network.

10.6 Filter Quick-Reference Card

Filter Reference

Goal	Filter
All Modbus	<code>modbus</code>
Modbus writes	<code>modbus.func_code == 5 </code> <code>modbus.func_code == 6 </code> <code>modbus.func_code == 15 </code> <code>modbus.func_code == 16</code>
Modbus exceptions	<code>modbus.exception_code > 0</code>
All DNP3	<code>dnp3</code>
DNP3 Operate	<code>dnp3.al.func == 4</code>
DNP3 unsolicited	<code>dnp3.ctl.uns == 1</code>
All OPC UA	<code>opcua</code>
TCP SYN (scan detect)	<code>tcp.flags.syn==1 &&</code> <code>tcp.flags.ack==0</code>
Retransmissions	<code>tcp.analysis.retransmission</code>
Large frames	<code>frame.len > 1400</code>
ARP only	<code>arp</code>
Broadcast	<code>eth.dst ==</code> <code>ff:ff:ff:ff:ff:ff</code>
Traffic to/from host	<code>ip.addr == X.X.X.X</code>
Traffic FROM host	<code>ip.src == X.X.X.X</code>
Specific TCP stream	<code>tcp.stream == N</code>
Zero-window	<code>tcp.analysis.zero_window</code>
HTTP (unexpected)	<code>http</code>

10.7 tshark Quick-Reference Card

Quick Reference

Command	Purpose
<code>tshark -D</code>	List interfaces
<code>tshark -i eth0 -w out.pcap</code>	Live capture to file
<code>tshark -r in.pcap -Y "modbus"</code>	Read + filter
<code>tshark -r in.pcap -T fields -e ip.src</code>	Extract field values
<code>tshark -r in.pcap -q -z io,phs</code>	Protocol hierarchy
<code>tshark -r in.pcap -q -z conv,tcp</code>	TCP conversations
<code>tshark -r in.pcap -q -z expert</code>	Expert info summary
<code>ssh host "tshark -i eth0 -w -" wireshark -k -i -</code>	Remote capture
<code>tshark -i eth0 -b filesize:102400 -b files:10 -w ring.pcap</code>	Ring buffer
<code>editcap -c 10000 big.pcap chunk.pcap</code>	Split PCAP
<code>capinfos capture.pcap</code>	File summary

Recommended Reading

- **Wireshark Network Analysis** — Laura Chappell. The authoritative reference. 1,000+ pages. Read chapters on TCP analysis and protocol troubleshooting.
- **The Practice of Network Security Monitoring** — Richard Bejtlich. Chapter on full-packet capture applies directly to ICS monitoring.
- **Industrial Network Security** — Eric Knapp & Joel Langill. ICS-specific threat model and network architecture.
- **DNP3 Specification** (IEEE 1815-2012). The authoritative source for DNP3 frame structure.
- **Modbus Specification** (modbus.org). Free download. The MBAP header definition is in Part 1.
- **OPC UA Specification Part 6** (opcfoundation.org). Mappings specification covers binary transport and the Hello/Acknowledge handshake.
- **ICS-CERT Advisories** (cisa.gov/ics). Current vulnerability reports for ICS protocols and devices.

Certification Context

Wireshark proficiency is tested or valued in the following credentials relevant to SCADA engineers:

Certification	Body	Wireshark Relevance
GICSP	GIAC	ICS/SCADA security; network analysis directly tested
GRID	GIAC	ICS defense; packet analysis in incident response modules
GCIP	GIAC	Critical infrastructure; protocol analysis concepts
ISA/IEC 62443	ISA	Network segmentation; capture-based verification
CEH	EC-Council	General packet analysis; Wireshark in network recon modules
CCST (ISA)	ISA	Networking fundamentals; protocol identification

The best Wireshark practice is your own production traffic.
The second best is replaying PCAPs.
The worst is never looking at the wire at all —
which is what 80% of engineers do.